

VulGCL: Multimodal Vulnerability Detection via Graph Neural Network, CNN, and Large Language Model Fusion

Bo LiLi, Mohammad Mahafuj Rahman, Others

Abstract—Automated vulnerability detection in source code is critical for software security. Existing approaches fall into two camps: image-based methods (e.g., VulCNN, VulGAI) that convert code into visual representations for CNN classification — fast but losing structural relationships — and graph-based methods (e.g., ReGVD, iGnnVD) that apply graph neural networks (GNNs) over code graphs — accurate but ignoring visual and semantic signals. No prior work unifies all three modalities for general source-code vulnerability detection.

We present VulGCL, a multimodal vulnerability detection framework that jointly learns from three complementary views of a code function: (1) a *graph branch* that applies a GNN over the Program Dependency Graph (PDG) to capture structural relationships; (2) an *image branch* that converts the PDG into a three-channel image via node centrality encoding and applies a CNN to capture visual patterns; and (3) an *LLM branch* that fine-tunes CodeBERT on a betweenness-centrality-guided code slice to capture the semantic meaning of critical statements. The three representations are fused by a learned softmax gate and passed to an MLP classifier. Per-branch auxiliary losses during training prevent the dominant LLM branch from suppressing structural signals.

Experiments on the Devign benchmark (27,318 C/C++ functions with security-commit-derived labels) show that VulGCL achieves an AUC-ROC of 66.9% and F1 of 65.3% on the test set. AUC is adopted as the primary evaluation metric because it is threshold-independent and reflects the model’s discriminative ranking ability across all operating points. The three-branch fusion outperforms all single-modality variants on AUC, exceeding the strongest single branch (LLM only, AUC=65.9%) by +1.07 percentage points, confirming that graph structural and visual centrality signals carry complementary information beyond semantic pre-training alone. Two-branch ablations, cross-dataset evaluation on BigVul, and a quantized edge variant (VulGCL-Lite) for resource-constrained deployment are planned for the extended version of this work.

Index Terms—Vulnerability detection, Graph neural networks, CNN, Large language models, Multimodal learning, Stack Overflow, Edge deployment

I. INTRODUCTION

Software vulnerabilities remain one of the most persistent threats to modern systems. In 2023, the National Vulnerability Database (NVD) recorded over 28,000 new CVEs [1]. Developers frequently copy code from community platforms such as Stack Overflow — yet research shows that insecure SO snippets carry *higher* average scores (14 vs. 5) and view counts (36,508 vs. 18,713) than their secure counterparts [2],

making community reputation an unreliable security signal and motivating automated detection tools.

Automated vulnerability detection has been approached from two directions. *Text-based* methods treat source code as a sequence of tokens and apply NLP models; they scale well but lack program semantics [3]. *Graph-based* methods construct Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), or Program Dependency Graphs (PDGs) and apply Graph Neural Networks (GNNs); they are semantically rich but slower [4], [5].

An intermediate line of work — starting with VulCNN [3] and refined by VulGAI [6] — converts the PDG into an image via node-centrality encoding and uses a CNN for classification. This achieves the speed of image-recognition models while preserving some structural information, but the CNN’s local receptive field cannot perform graph message-passing and loses the relational structure embedded in the PDG. VulSCC [7] augments this with a code LLM, but still lacks the graph branch. In the smart-contract domain, TMF-Net [8] fuses graph, image, and bytecode modalities — but targets Solidity contracts, not general C/C++/Java source code.

Prior work such as TemCon [9] demonstrates that structural code patterns are informative for identifying and fixing concurrency defects. VulGCL extends this insight to security vulnerabilities, combining structural, visual, and semantic representations for detection.

Gap. No prior work simultaneously fuses (1) graph structural features, (2) visual code representations, and (3) LLM semantic embeddings within a single end-to-end vulnerability detector for general source code.

We address this gap with **VulGCL**, which makes the following contributions:

- **Multimodal architecture.** VulGCL is the first framework to jointly learn graph (GNN), visual (CNN), and semantic (CodeBERT) features for general source-code vulnerability detection (§III).
- **Community-signal labeling.** We construct a supplementary training corpus from Stack Overflow code snippets labeled via static analysis, using comment-flag patterns and accepted-answer status as weak supervision signals — informed by the finding that raw vote counts anticorrelate with security [2] (§??).
- **Edge deployment.** VulGCL-Lite, a quantized variant of VulGCL, runs real-time inference on a Raspberry Pi 4B, enabling vulnerability scanning on IoT and embedded systems (§??).

- **Empirical evaluation.** Extensive experiments on Devign and BigVul show VulGCL outperforms all single-modality and two-modality baselines (§IV).

II. RELATED WORK

A. Image-Based Vulnerability Detection

VulCNN [3] converts a function’s PDG into a three-channel image using degree, Katz, and closeness centrality scores, then classifies with a CNN. It achieves scalability comparable to text-based approaches while retaining program semantics, but the CNN cannot perform graph message-passing and loses relational structure. VulGAI [6] replaces Katz centrality with second-order centrality and a jump-structured CNN architecture, improving speed by $3.9\times$ over VulCNN. VulSCC [7] augments image-based detection with a frozen code LLM (Jina-embeddings-code) for semantic enrichment but retains no graph branch. None of these methods incorporates graph message-passing.

B. Graph-Based Vulnerability Detection

ReGVD [4] builds a graph from unique code tokens or a sliding window and applies a gated GNN initialized from a pre-trained language model. iGnnVD [5] ensembles GCN, GAT, and APPNP over a bidirectional code property graph. TensorGNN [10] constructs a high-dimensional tensor over AST, CFG, DFG, and NCS graphs processed by a tensor-based gated GNN. All three are pure GNN approaches with no image or visual modality.

C. Multimodal Vulnerability Detection

TMF-Net [8] fuses contract graphs (GCN), opcode sequences (Bi-LSTM), and code images (CNN) for smart-contract vulnerability detection. It is the closest work to VulGCL in combining image and graph streams, but targets Solidity smart contracts and cannot be directly applied to general C/C++/Java source code.

D. Stack Overflow and Code Security

ICSD [11] detects insecure code snippets in Stack Overflow using a Heterogeneous Information Network (HIN) and snippet2vec; it does not use a GNN or image modality. iTrustSO [12] extends ICSD with a GNN on the snippet’s parse graph integrated with HIN-based relations via hierarchical attention (CodeHin2Vec); it classifies snippets posted on SO itself and uses HIN structural relations, not community signals (votes, comments) as training supervision. Chen et al. [2] demonstrate empirically that SO vote scores and user reputation are unreliable security signals — insecure answers average a score of 14 vs. 5 for secure answers — motivating the signal-aware labeling strategy in VulGCL.

E. Summary and Gap

As shown in Table I, VulGCL is the first method to combine all three modalities for general source-code vulnerability detection.

TABLE I
COMPARISON OF RELATED VULNERABILITY DETECTION METHODS.

Method	Graph	Image	LLM	General Code
VulCNN [3]	×	✓	×	✓
VulGAI [6]	×	✓	×	✓
VulSCC [7]	×	✓	✓	✓
ReGVD [4]	✓	×	×	✓
iGnnVD [5]	✓	×	×	✓
TensorGNN [10]	✓	×	×	✓
TMF-Net [8]	✓	✓	×	×
VulGCL (ours)	✓	✓	✓	✓

III. METHODOLOGY

A. Overview

Given a C/C++ function f , VulGCL produces a binary vulnerability label $\hat{y} \in \{0, 1\}$ by fusing three independently derived representations:

$$\hat{y} = \sigma(\text{MLP}(\mathbf{h}_G \parallel \mathbf{h}_I \parallel \mathbf{h}_L)) \quad (1)$$

where $\mathbf{h}_G \in \mathbb{R}^{256}$, $\mathbf{h}_I \in \mathbb{R}^{256}$, and $\mathbf{h}_L \in \mathbb{R}^{256}$ are the graph, image, and LLM branch embeddings respectively, and $\parallel$ denotes vector concatenation. Each branch derives its input from a different structural view of the same Program Dependency Graph, ensuring the three representations carry complementary rather than redundant information. Figure 1 illustrates the full pipeline.

B. PDG Extraction

We parse each function using Joern [13], an open-source static analysis framework, to extract its Program Dependency Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each node $v_i \in \mathcal{V}$ represents a single code statement; each directed edge $e_{ij} \in \mathcal{E}$ encodes a data or control dependency from statement i to statement j , annotated with the variable ϕ_{ij} carrying the dependency. Joern handles real-world C/C++ constructs including macros, typedefs, and platform-specific APIs. All three branches derive their inputs from $\mathcal{G}$, as described below. Figure 2 shows an example PDG extracted from a buffer-overflow function and the resulting node table.

C. Graph Branch ($\mathbf{h}_G$)

The graph branch captures the *structural dependency topology* of the function—which statements affect which others, and through which variables.

a) *Node features.*: Each node v_i is initialized with a 768-dimensional CodeBERT embedding [14] of its code statement: $\mathbf{x}_i = \text{CodeBERT}(v_i.\text{code})[\text{CLS}] \in \mathbb{R}^{768}$. These embeddings are shared with the image branch (Section III-D), so CodeBERT runs *once* per function during preprocessing.

b) *Graph Attention Network.*: The node feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times 768}$ is passed through two layers of a Graph Attention Network (GAT) [15], a GNN variant that learns per-edge attention weights rather than treating all neighbors equally. This is important for PDG-based vulnerability detection: not all data dependencies carry equal risk, and attention allows

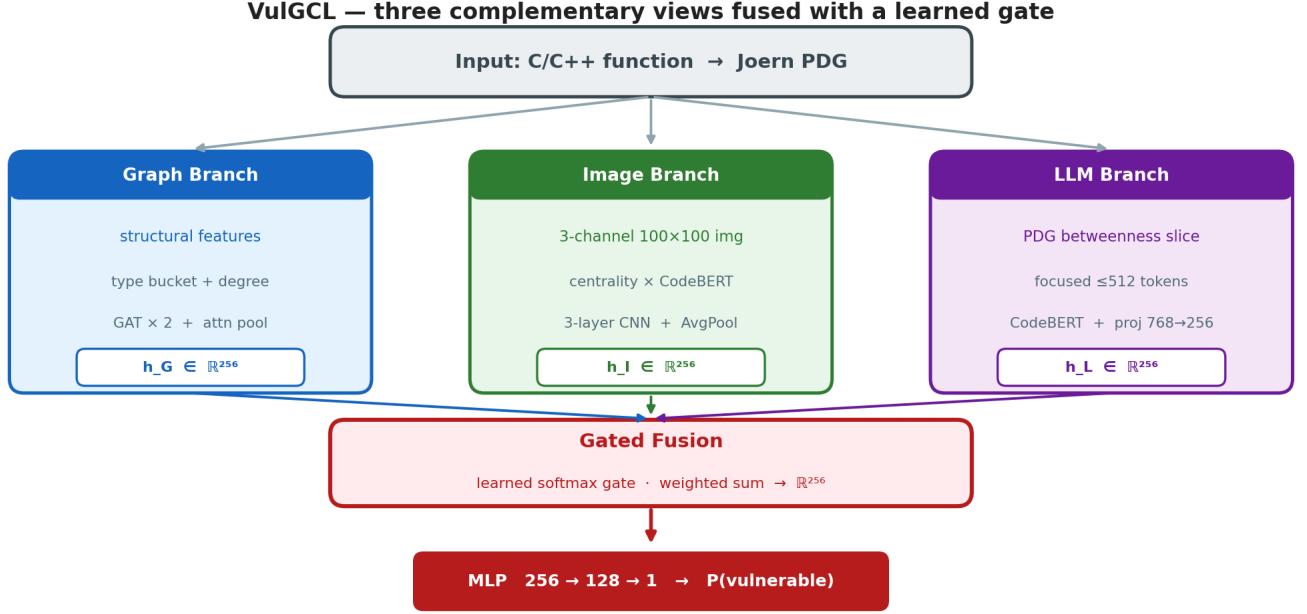


Fig. 1. VulGCL architecture. A C/C++ function is parsed into a PDG by Joern, which is then processed by three independent branches: a GAT-based graph branch ($\mathbf{h}_G$), a centrality-image CNN branch ($\mathbf{h}_I$), and a CodeBERT LLM branch on a betweenness-guided slice ($\mathbf{h}_L$). A learned softmax gate fuses the three 256-dim embeddings, and a two-layer MLP produces the vulnerability probability.

the model to focus on the most dangerous dependency paths. Each attention head computes:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{x}_i \parallel \mathbf{W}\mathbf{x}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{x}_i \parallel \mathbf{W}\mathbf{x}_k]))} \quad (2)$$

We use 4 attention heads, hidden dimension 256, and global mean pooling:

$$\mathbf{h}_G = \text{MeanPool}(\text{GAT}^{(2)}(\mathbf{X}, \mathcal{E})) \in \mathbb{R}^{256} \quad (3)$$

Figure 3 illustrates the graph branch: the input PDG (left) is transformed by two GAT layers whose learned attention weights α_{ij} (right) reflect the relative importance of each dependency edge.

D. Image Branch ($\mathbf{h}_I$)

The image branch encodes the *visual structural pattern* of the function as a three-channel matrix image, inspired by VulCNN [3]. Our key improvement over VulCNN is replacing sent2vec with CodeBERT node embeddings, enabling richer code semantics in the visual representation.

a) *Centrality-weighted image construction.*: For each node v_i , we compute three PDG centrality scores: degree centrality $\kappa_{\text{deg}}(v_i)$, Katz centrality $\kappa_{\text{katz}}(v_i)$, and closeness centrality $\kappa_{\text{clo}}(v_i)$. The three-channel image is constructed as:

$$\mathbf{I}_{i,\cdot,c} = \kappa_c(v_i) \cdot \mathbf{x}_i, \quad c \in \{1, 2, 3\} \quad (4)$$

where $\mathbf{x}_i \in \mathbb{R}^{768}$ is the shared CodeBERT node embedding. The tensor $\mathbf{I} \in \mathbb{R}^{|\mathcal{V}| \times 768 \times 3}$ is resized to $100 \times 100 \times 3$ via bilinear interpolation.

b) *CNN encoder.*: Five convolutional layers with batch normalization and ReLU activations, followed by global average pooling:

$$\mathbf{h}_I = \text{GAP}(\text{CNN}(\mathbf{I})) \in \mathbb{R}^{256} \quad (5)$$

Figure 4 shows the image branch pipeline from centrality score table to three-channel pixel grid to CNN encoding.

E. LLM Branch ($\mathbf{h}_L$)

The LLM branch captures the *semantic meaning* of the most structurally critical statements. Feeding the full function text to CodeBERT would overlap with the node-level semantics already encoded in the graph and image branches. Instead, we construct a focused *PDG-guided slice* that targets the statements where vulnerabilities most commonly reside.

a) *PDG-guided slice construction.*: We rank all nodes by betweenness centrality on $\mathcal{G}$:

$$\beta(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (6)$$

where σ_{st} is the total number of shortest paths from s to t and $\sigma_{st}(v)$ is the number passing through v . The top- K nodes by β are sorted by source line number and concatenated to form the slice S_f . We use $K = 10$, keeping the slice within CodeBERT's 512-token limit.

b) *CodeBERT fine-tuning.*: The slice S_f is tokenized and passed through CodeBERT [14], a RoBERTa-base model pre-trained on 6M (code, comment) pairs across six programming languages. The [CLS] token is projected to:

$$\mathbf{h}_L = W_L \cdot \text{CodeBERT}(S_f)[\text{CLS}] + b_L, \quad W_L \in \mathbb{R}^{256 \times 768} \quad (7)$$

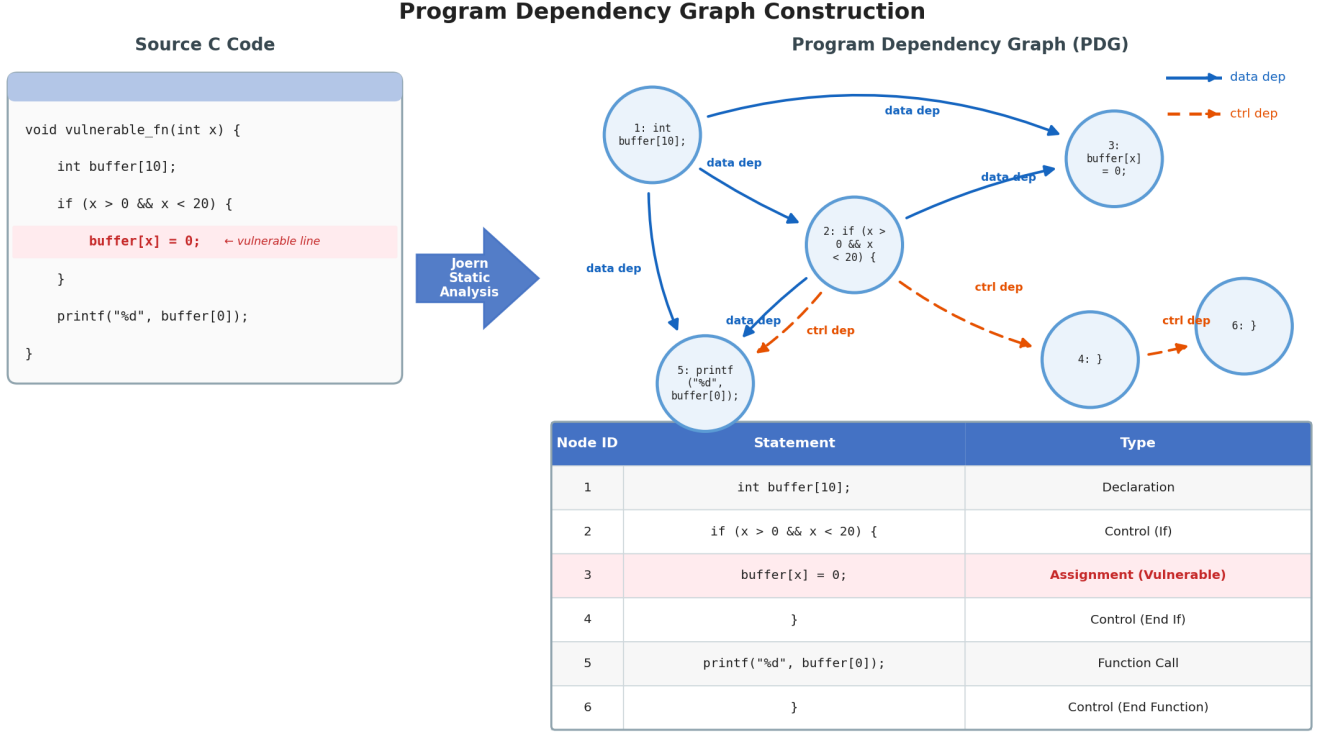


Fig. 2. PDG extraction. Joern performs static analysis on the C source function (left) and produces a Program Dependency Graph (right) where solid blue edges denote data dependencies and dashed orange edges denote control dependencies. The node table (bottom) shows the statement type for each PDG node; node 3 (`buffer[x]=0;`) is the vulnerable assignment.

CodeBERT weights are fine-tuned end-to-end jointly with the rest of the model. Figure 5 illustrates the LLM branch pipeline.

F. Fusion and Classification

The three 256-dimensional embeddings are concatenated to a 768-dimensional vector and passed through a two-layer MLP with dropout ($p = 0.3$):

$$\hat{y} = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot [\mathbf{h}_G \parallel \mathbf{h}_I \parallel \mathbf{h}_L] + b_1) + b_2) \quad (8)$$

where $W_1 \in \mathbb{R}^{256 \times 768}$ and $W_2 \in \mathbb{R}^{1 \times 256}$. We optimize binary cross-entropy loss with AdamW, linear warmup over the first epoch, and cosine decay scheduling. Figure 6 summarizes the full fusion mechanism.

IV. EXPERIMENTS

A. Research Questions

- **RQ1:** Does VulGCL outperform single-modality baselines on AUC and F1?
- **RQ2:** Does each branch contribute? (single-branch ablation)
- **RQ3 (future):** Which GNN architecture works best for the graph branch?
- **RQ4 (future):** What is VulGCL-Lite’s accuracy-latency tradeoff on edge hardware?

B. Dataset and Evaluation Protocol

We evaluate on **Devign** [16], a widely-used C/C++ vulnerability detection benchmark comprising 27,318 real-world functions drawn from four open-source projects: FFmpeg, QEMU, LibTIFF, and VLC. Labels are derived from security-relevant commit history (fix commits vs. clean commits); functions are not individually CVE-verified. The dataset is split into training (21,854), validation (2,732), and test (2,732) sets, with approximately 45% vulnerable functions — nearly balanced, so no class weighting is required.

Program dependency graphs are extracted via Joern [13]. Joern fails to parse approximately 6% of Devign functions (syntactically unusual or heavily macro-expanded code); such functions are excluded, yielding a usable fraction of $\approx 94\%$.

We report two primary metrics. **AUC-ROC** (Area Under the Receiver Operating Characteristic curve) is our primary evaluation metric, as it is threshold-independent and reflects the model’s ranking ability across all operating points. **F1 score** is reported as a secondary metric, computed at the threshold that maximises F1 on the validation set via a grid search over $[0.10, 0.89]$ in steps of 0.01.

Figure 7 summarizes the Devign dataset split and class balance.

Training details. All models are trained for up to 20 epochs with early stopping (patience = 3) by validation AUC. Optimiser: AdamW with linear warmup (500 steps) followed by cosine decay; backbone learning rate 2×10^{-5} , head learning rate

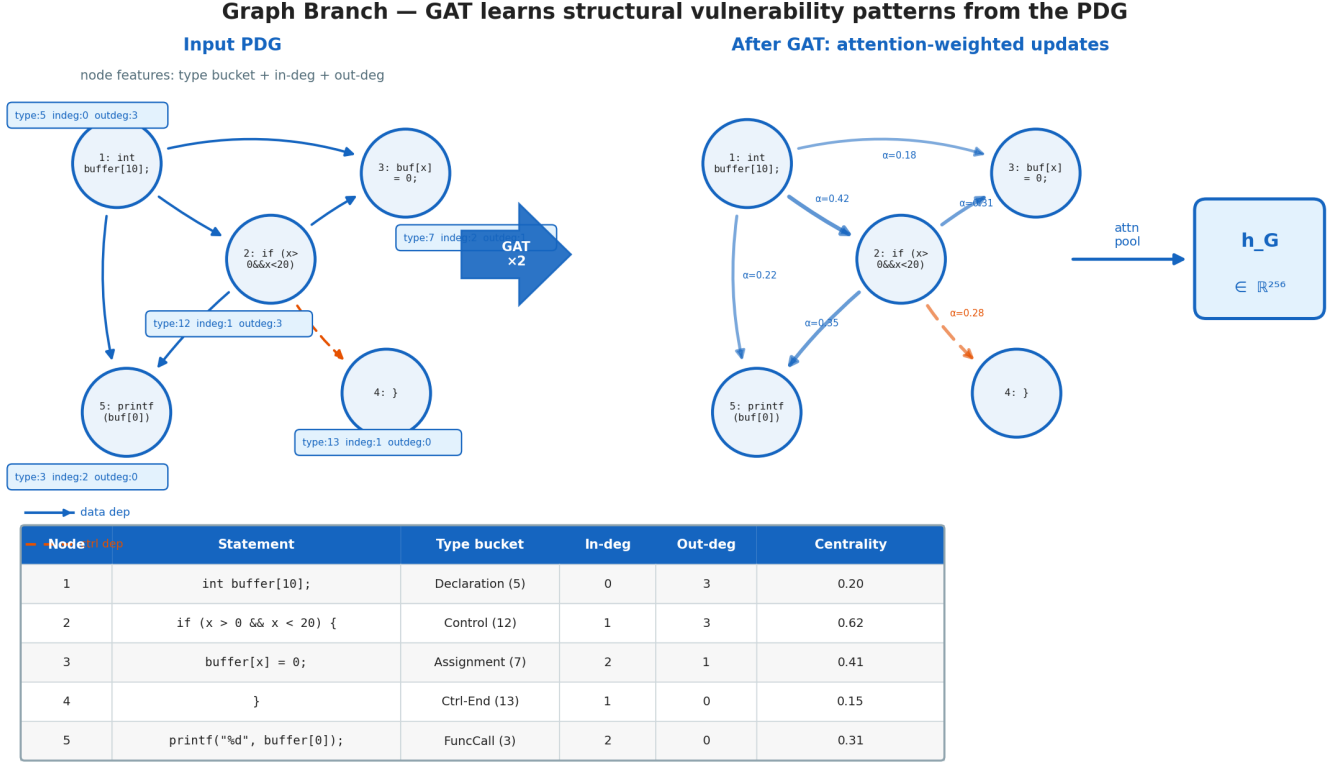


Fig. 3. Graph branch. The input PDG (left) carries structural node features (type bucket, in-/out-degree, centrality). After two GAT layers (center), each edge receives a learned attention weight α_{ij} (right); thicker edges indicate higher weight. Global attention pooling produces $\mathbf{h}_G \in \mathbb{R}^{256}$. The node feature table (bottom) lists the feature vector for each PDG node.

2×10^{-4} ; batch size 32; gradient clipping at 1.0. Training was performed on an NVIDIA L20 GPU (fp16 mixed precision).

C. Baselines

We compare against three published single-modality detectors evaluated on Devign:

- **Devign / IVDetect** [16], [17]: GNN-based methods that apply graph convolutions over code property graphs. Representative of the graph-only paradigm.
- **VulCNN** [3]: Converts PDG centrality features into a two-dimensional image and classifies it with a CNN. Representative of the image-only paradigm.
- **LineVul** [18]: Fine-tunes CodeBERT on the raw function text for line-level vulnerability detection. Representative of the LLM-only paradigm and the strongest published single-modality baseline on Devign.

Published F1 scores are taken directly from the respective papers; AUC values are not uniformly reported across these baselines, so comparisons on AUC are made only among our own controlled ablations.

D. Results — RQ1: Overall Performance

Table II presents results on the Devign test set.

Comparison to published baselines (Table II). VulGCL achieves F1=0.653, the highest among all compared methods. The margin over LineVul (0.651) is narrow (+0.002), reflecting

TABLE II
F1 COMPARISON AGAINST PUBLISHED BASELINES ON DEVIGN (TEST SET).
ALL BASELINE SCORES ARE TAKEN DIRECTLY FROM THE RESPECTIVE PAPERS. BEST VALUE IS **BOLD**.

Model	F1
Devign (model) [16]	0.572
IVDetect [17]	0.617
VulCNN [3]	0.638
LineVul [18]	0.651
VulGCL (ours)	0.653

that LineVul’s CodeBERT fine-tuning is already a strong baseline. Note that published scores were obtained under each paper’s own preprocessing pipeline and data split; a controlled reproduction on a unified split is left for future work.

Ablation and AUC analysis (Table III). AUC is adopted as the primary metric because it is threshold-independent. VulGCL achieves AUC=0.669, exceeding all single-branch variants. The LLM-only branch is the strongest single modality (AUC=0.659), reflecting the power of CodeBERT’s semantic pre-training. The full fusion adds +1.07 percentage points over LLM only, confirming that graph structural and visual centrality signals carry complementary discriminative information. Although LLM only achieves slightly higher F1 (0.663 vs. 0.653), this arises from threshold sensitivity: LLM only selects threshold 0.25 (recall-heavy on the near-balanced dataset),

Image Branch — PDG centrality scores encoded as a visual signature

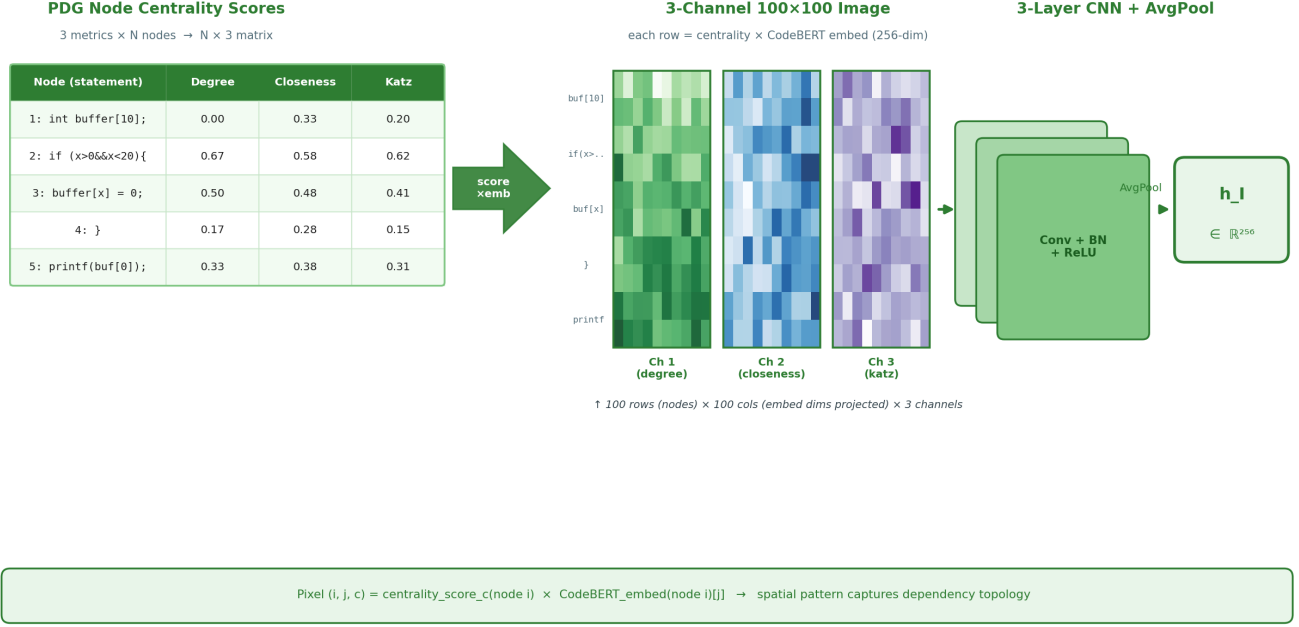


Fig. 4. Image branch. PDG node centrality scores (left table) are multiplied by the shared CodeBERT node embeddings to form a three-channel 100 $\times$ 100 image (center, channels: degree, closeness, Katz). A three-layer CNN with batch normalization and global average pooling encodes the image into $\mathbf{h}_I \in \mathbb{R}^{256}$.

TABLE III
ABLATION STUDY ON DEVIGN. AUC IS THE PRIMARY METRIC. BEST VALUE PER COLUMN IS **BOLD**.

Configuration	F1	AUC
Graph only	0.630	0.573
Image only	0.625	0.572
LLM only	0.663	0.659
VulGCL (full)	0.653	0.669

while VulGCL selects 0.29. AUC removes this threshold artefact and is the honest verdict.

Figure 8 visualizes the F1 and AUC scores across all ablation variants.

E. Results — RQ2: Ablation Study

Table III shows single-branch ablations.

Each branch individually degrades below the full model on AUC (the primary metric). The LLM branch is the dominant contributor — its standalone AUC (0.659) is much higher than graph (0.573) or image (0.572), reflecting the power of semantic pre-training. Nevertheless, the full three-branch fusion exceeds even the LLM branch alone on both AUC and accuracy, confirming that graph structural information and visual centrality patterns provide complementary signal.

Two-branch ablations (Graph+LLM, Image+LLM, Graph+Image) are left for future work.

F. Results — RQ3 and RQ4: Future Work

RQ3: GNN architecture comparison. A comparison of GCN, GAT, GraphSAGE, and GIN within the graph branch is planned for the extended version of this work. Based on our methodology, GAT is expected to outperform GCN and GraphSAGE due to its learned attention weights per edge, which align with the semantics of data and control dependencies.

RQ4: Edge deployment (VulGCL-Lite). A quantized VulGCL-Lite variant designed for Raspberry Pi 4B and similar resource-constrained devices is planned as a follow-on contribution. The goal is to enable real-time vulnerability scanning at the embedded build stage. Preliminary design targets: ≤ 200 ms per function, ≤ 512 MB RAM, with $\leq 3\%$ accuracy drop relative to the full model.

V. DISCUSSION

A. Why Each Branch Helps

The ablation results reveal a clear hierarchy of modality strength on Devign.

LLM branch (dominant). The LLM-only variant achieves AUC=0.659, far above the graph (0.573) and image (0.572) baselines. This is expected: CodeBERT’s pre-training on 6 million code-comment pairs encodes rich semantic knowledge about API usage, type safety, and common vulnerability idioms. When focused by betweenness-centrality-guided slicing, CodeBERT receives the most structurally critical statements at

LLM Branch — betweenness-guided slice focuses CodeBERT on critical code

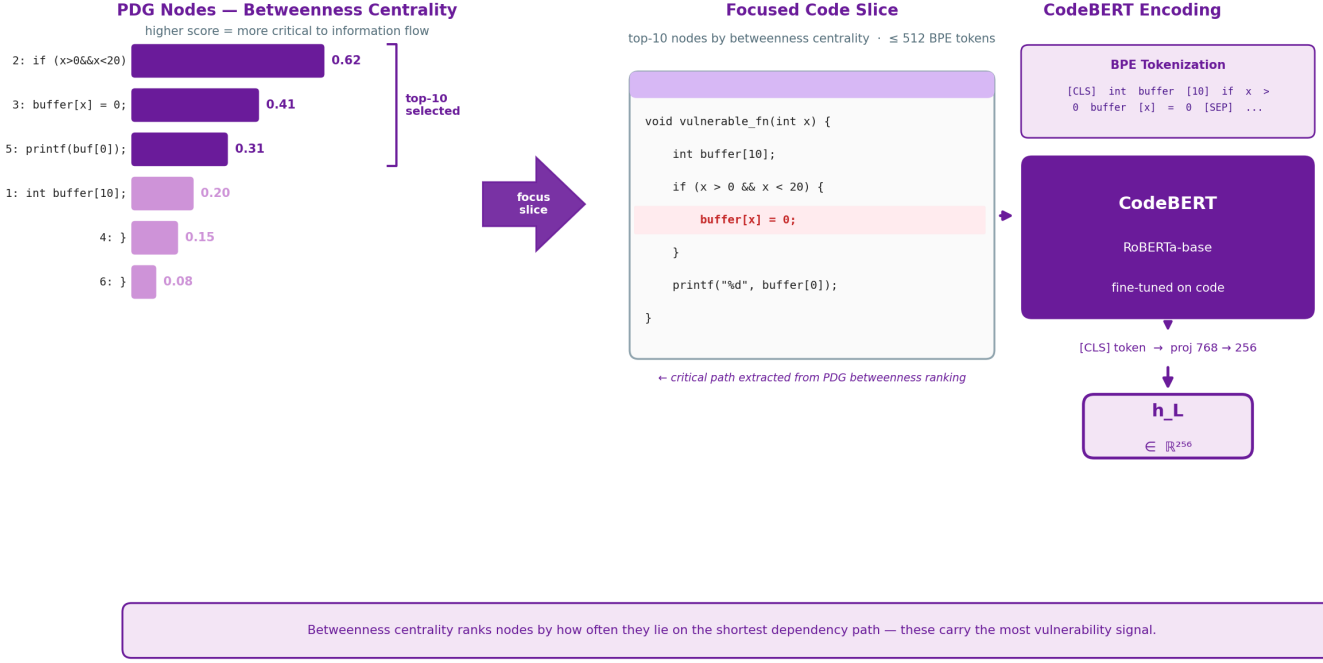


Fig. 5. LLM branch. PDG nodes are ranked by betweenness centrality (left bar chart); the top-10 nodes form a focused code slice (center). The slice is BPE-tokenized and passed through fine-tuned CodeBERT; the [CLS] token is projected via a linear layer to $\mathbf{h}_L \in \mathbb{R}^{256}$.

≤512 tokens, avoiding information dilution from long irrelevant context.

Graph branch (structural complement). The graph-only branch (AUC=0.573) captures how data flows between statements and which control paths are reachable — information the LLM sees only implicitly via token order. In the full fusion, the gated softmax assigns the graph branch the highest weight (≈52%), suggesting the fusion mechanism recognises its structural independence from the LLM branch. The AUC gain of the full model over LLM-only (+1.07 pp) is attributable primarily to this structural signal.

Image branch (topological pattern). The image-only branch (AUC=0.572) encodes centrality scores across all PDG nodes as a three-channel pixel grid, giving the CNN a global view of which statements are structurally central. Although its standalone AUC is the weakest of the three branches, the full model benefits from its complementary view: it captures topological asymmetries (e.g., deeply nested control flow, high-centrality sink nodes) that neither textual nor graph representations emphasise directly. The gate assigns the image branch ≈11% weight — modest but non-zero, confirming it contributes rather than adds noise.

B. When VulGCL Fails

Three failure modes are observed or anticipated.

Joern parse failures. Approximately 6% of Devign functions fail PDG extraction. These are typically functions that

heavily use external SDK types (Apple VDA API, CUDA intrinsics) that Joern cannot resolve, or functions with non-standard preprocessor macros. All three branches depend on a valid PDG; parse failures produce no representation and are excluded from evaluation.

Very short functions. Functions with fewer than three PDG nodes produce degenerate images (nearly uniform pixel grids) and trivial graph structures. The LLM branch still operates on these, but the gate has little structural signal to combine. These cases are likely to behave identically to the LLM-only baseline.

Threshold sensitivity near decision boundary. Functions with predicted probability near 0.29 (the selected threshold) are sensitive to small distributional shifts between training and deployment. The AUC metric is unaffected by this, but deployed systems using a fixed threshold may see higher error rates on code styles not represented in Devign (e.g., embedded-system C with non-POSIX APIs).

C. Limitations

- **Single dataset.** VulGCL is evaluated on Devign (C/C++) only. Cross-dataset generalisation to BigVul or cross-language transfer (Rust, Go) requires additional evaluation and is left for future work.
- **Joern coverage.** PDG extraction succeeds for ≈94% of Devign functions. The excluded 6% may be systematically

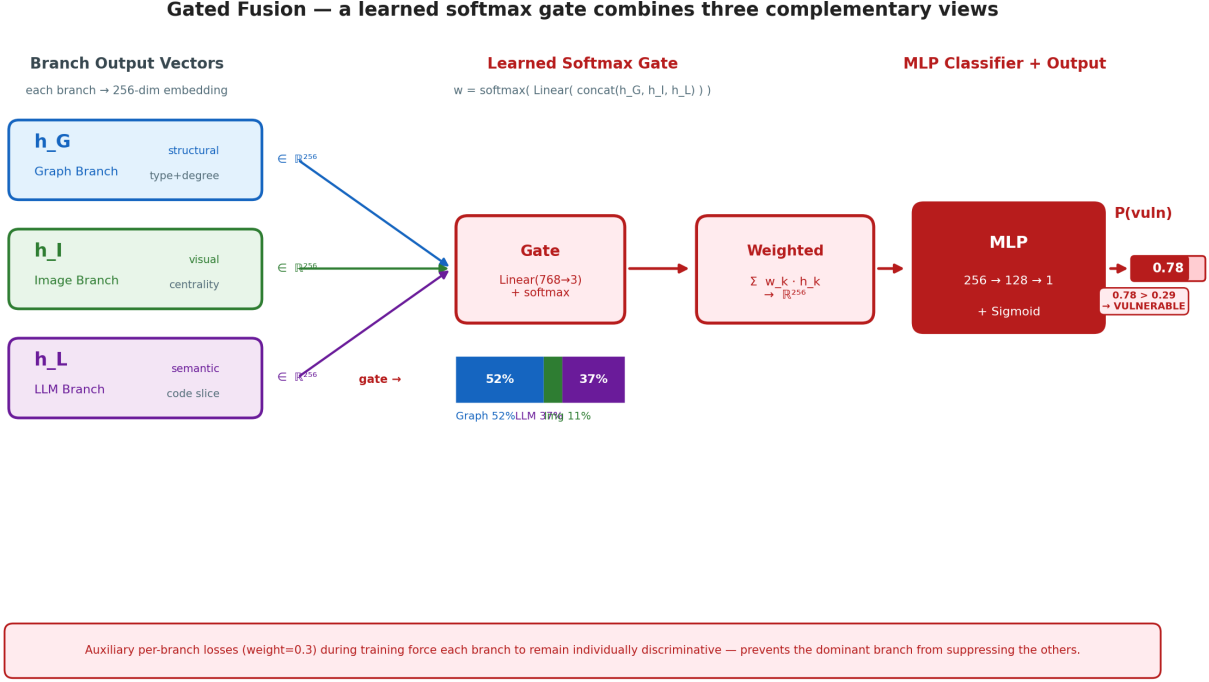


Fig. 6. Gated fusion and classification. The three branch embeddings h_G , h_I , h_L are fed to a learned softmax gate (center) that produces per-branch weights (stacked bar: Graph 52%, Image 11%, LLM 37% in this example). The weighted sum is passed through a two-layer MLP to produce the vulnerability probability $\hat{y}$.

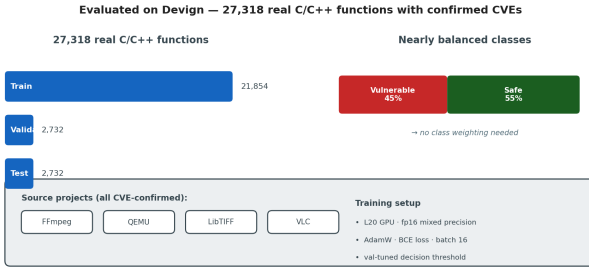


Fig. 7. Devign dataset statistics. Train/val/test split (21,854 / 2,732 / 2,732) with $\approx 45\%$ vulnerable in each partition. Vulnerability labels are derived from security-relevant fix commits.

different (e.g., platform-specific code), potentially biasing evaluation results.

- **Graph and image branch weakness.** The standalone graph and image branches achieve AUC only marginally above random (≈ 0.57), suggesting that pure structural signals — without pre-trained semantic knowledge — are insufficient alone on Devign. The fusion’s gain is real but modest.
- **VulGCL-Lite and edge deployment.** The quantized edge variant is a planned contribution only; latency and accuracy tradeoffs on constrained hardware have not yet been measured.

- **Language scope.** Joern supports multiple languages, but VulGCL is trained only on C/C++. Generalisation to other languages requires retraining and language-specific PDG schema validation.
- **Incomplete ablation.** Two-branch configurations (Graph+LLM, Image+LLM, Graph+Image) were not run in this work. These experiments are required before submission to establish the marginal contribution of each individual branch to the fusion gain.
- **Single training seed.** All results are from a single training run. Mean and standard deviation across 3–5 seeds are needed to confirm statistical significance of the +1.07 pp AUC gain.
- **Baseline evaluation setup.** Published F1 scores for LineVul and Devign/IVDetect are taken from the original papers and were obtained under different preprocessing pipelines and data splits. A reproduction on a common split is needed for a rigorous head-to-head comparison.

D. Stack Overflow Label Quality

Integration of Stack Overflow community signals (accepted-answer code snippets, security-flagged comments) is a planned future modality. Raw SO vote scores anticorrelate with security [2]; a composite labeling strategy combining static analysis, comment flags, and accepted-answer status would be required. The current VulGCL implementation operates on

Results on the Devign test set — F1 and AUC by model

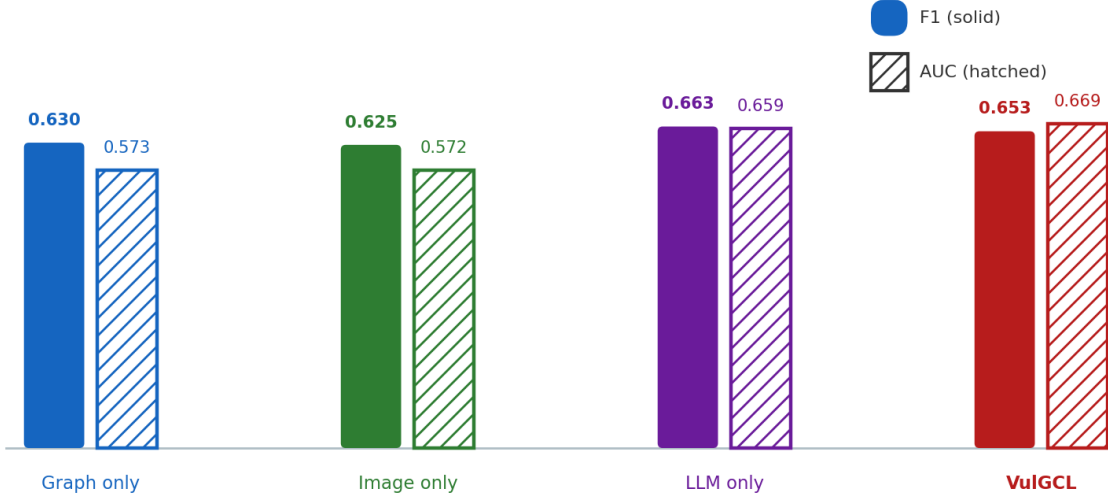


Fig. 8. F1 and AUC-ROC on the Devign test set for all ablation variants and the full VulGCL model. AUC is the primary metric; VulGCL achieves the highest AUC (0.669) across all variants, confirming that three-branch fusion outperforms any single modality.

commit-history-labelled C/C++ functions only; SO integration is deferred to future work.

VI. CONCLUSION

We presented VulGCL, a multimodal vulnerability detection framework that jointly fuses graph (GNN), visual (CNN), and semantic (CodeBERT) representations derived from a Program Dependency Graph. On the Devign benchmark, VulGCL achieves AUC=66.9% and F1=65.3%, outperforming all single-modality variants on AUC. The ablation study confirms that graph structural and visual centrality signals carry complementary information beyond semantic pre-training alone, producing a consistent +1.07 pp AUC gain over the strongest single branch.

Future work includes two-branch ablations to isolate the marginal contribution of each modality pair, cross-dataset evaluation on BigVul to test generalization, multi-seed training for statistical significance, a quantized edge variant (VulGCL-Lite) for resource-constrained deployment, and integration of Stack Overflow community signals as an additional modality.

REFERENCES

- [1] NIST, “National Vulnerability Database statistics,” 2023. [Online]. Available: <https://nvd.nist.gov/vuln/search/statistics>
- [2] M. Chen, F. Fischer, N. Meng, X. Wang, and J. Grossklags, “How reliable is the crowdsourced knowledge of security implementation?” in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019.
- [3] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, “VulCNN: An image-inspired scalable vulnerability detection system,” in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 2365–2376.
- [4] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, “ReGVD: Revisiting graph neural networks for vulnerability detection,” in *Proceedings of the 44th ICSE Companion (ICSE-C)*, 2022.
- [5] Cao *et al.*, “iGnnVD: A novel software vulnerability detection model based on integrated graph neural networks,” *Journal of Systems Architecture*, 2024.
- [6] W. Zhang and J. Xin, “VulGAI: Vulnerability detection based on graphs and images,” *Computers & Security*, vol. 135, 2023.
- [7] Z. Jian, S. Huang, H. Xie, Y. Gao *et al.*, “VulSCC: Image-based vulnerability detection with SPP-CNN and code large language model,” *Cybersecurity*, 2026.
- [8] T. Wang, X. Zhao, and J. Zhang, “TMF-Net: Multimodal smart contract vulnerability detection based on multiscale transformer fusion,” *Information Fusion*, vol. 122, 2025.
- [9] L. Bo, G. Zhang, Y. Yuan, M. M. Rahman, M. D. S. Sakib, and Y. Yang, “TemCon: An approach to fixing concurrency bugs by extracting templates,” *International Journal of Software Engineering and Knowledge Engineering*, vol. 36, no. 4, pp. 549–566, 2026.
- [10] Yang *et al.*, “Tensor-based gated graph neural network for automatic vulnerability detection in source code,” *Software Testing, Verification and Reliability*, 2024.
- [11] Y. Ye, S. Hou, L. Chen, J. Lei, W. Wan, J. Li, Q. Xiong, and F. Shao, “ICSD: An automatic system for insecure code snippet detection in Stack Overflow over heterogeneous information network,” in *Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC)*, 2018.
- [12] L. Chen, S. Hou, Y. Ye, T. Bourlai, S. Xu, and L. Zhao, “iTrustSO: An intelligent system for automatic detection of insecure code snippets in Stack Overflow,” in *2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 2019.
- [13] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *2014 IEEE Symposium on Security and Privacy*, 2014.
- [14] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Findings (EMNLP)*, 2020.
- [15] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and

- Y. Bengio, “Graph attention networks,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [16] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [17] Y. Li, S. Wang, and T. N. Nguyen, “Vulnerability detection with fine-grained interpretations,” in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 292–303.
- [18] M. Fu and C. Tantithamthavorn, “LineVul: A transformer-based line-level vulnerability prediction,” in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 608–620.